NAME: KEERTHIKA S **ROLL NO: 231901024**

TITLE: EXP NO.: 8 FABRIC NETWORK DEPLOYMENT AND MANAGEMENT

1. AIM

To deploy, manage, monitor, and maintain a Hyperledger Fabric network by creating the network, verifying network components, deploying chaincode, monitoring Docker containers, and performing network shutdown and cleanup operations.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. THEORY

Hyperledger Fabric network deployment involves setting up the blockchain infrastructure required for enterprise applications. A Fabric network consists of peer nod es, ordering service, Certificate Authorities (CAs), channels, chaincode, and client applications. Network management includes: • Starting and stopping the network • Creating application channels • Deploying chaincode • Monitoring Docker containers • Managing peer and orderer services • Viewing network logs • Cleaning up Docker

containers and generated artifacts Proper deployment and management ensure reliable blockchain operations and simplify maintenance and troubleshooting.

5. PROCEDURE

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify Running Docker Containers

Display all active Docker containers.

```
docker ps
```

Step 4: Display Docker Images

```
docker images
```

Step 5: Check Network Status

Display all Docker containers including stopped containers.

```
docker ps -a
```

Step 6: Deploy Asset Transfer Chaincode

```
./network.sh deployCC
-ccl javascript
-ccn basic
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 7: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 8: Monitor Peer Logs

Display logs of the peer node.

```
docker logs peer0.org1.example.com
```

Step 9: Monitor Orderer Logs

```
docker logs orderer.example.com
```

Step 10: Verify the Channel

```
peer channel list
```

Step 11: Stop the Fabric Network

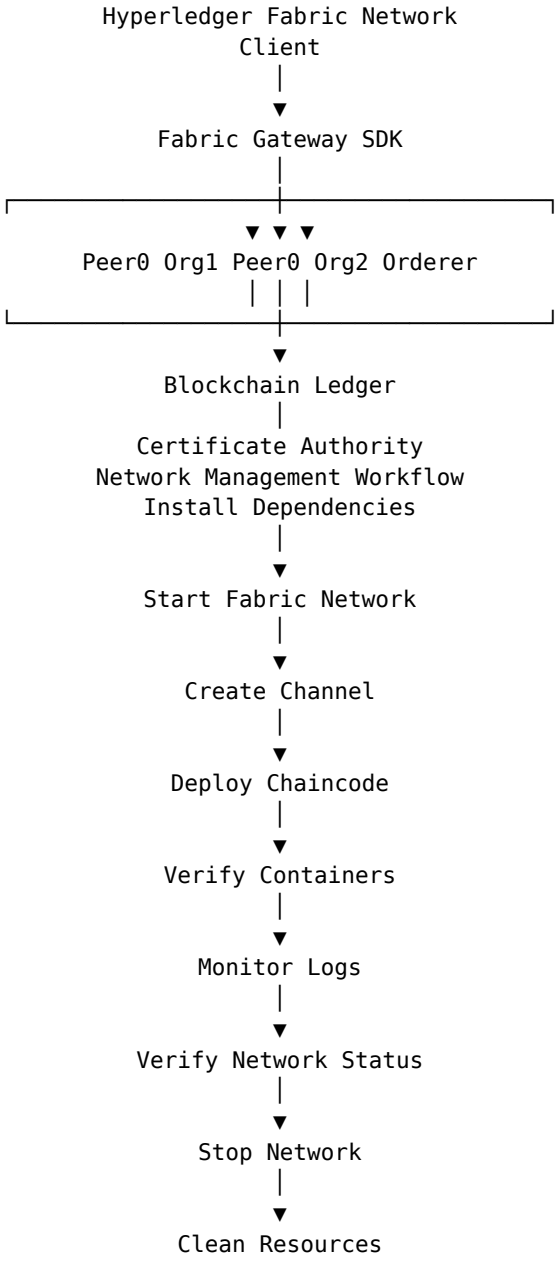
```
./network.sh down
```

Step 12: Clean Docker Resources (Optional)

Remove unused Docker containers, networks, and images.

```
docker system prune -f
```

6. ARCHITECTURE DIAGRAM



7. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	`cd ~/fabric-dev/fabric-samples/test-network`
Start network	`./network.sh up createChannel -ca`
Check containers	`docker ps`
Command	`docker images`
Check containers	`docker ps -a`
Deploy chaincode	`./network.sh deployCC`
Query ledger	`peer lifecycle chaincode queryinstalled`
View logs	`docker logs peer0.org1.example.com`
View logs	`docker logs orderer.example.com`
Verify channel	`peer channel list`
Stop network	`./network.sh down`
Command	`docker system prune -f`

8. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1

```
Experiment 8 - Fabric Output

$ Fabric Network Started Successfully
Docker Containers Running
Peer Nodes Active
Orderer Active
Certificate Authorities Running
Channel Created Successfully
Chaincode Installed Successfully
Network Verified Successfully
Fabric Network Stopped Successfully
```

Fig. 8.1 – Fabric output corresponding to the source experiment output.

9. RESULT

The Hyperledger Fabric network was successfully deployed and managed. The blockchain network components, including peer nodes , ordering service, Certificate Authorities, and application channel, were verified. Chaincode was deployed successfully, network status was monitored, and the Fabric network was shut down and cleaned up successfully.